

DEEPL - Lab 10

Graph Neural Networks

Nicolas Baskiotis - Stéphane Rivaud - Benjamin Piwowarski - Laure Soulier

2025-2026

Graphs are particular structures that cannot be processed sequentially (like text, time series, ...) and are not necessarily regular (like images, grids, ...). These structures are used in many application domains such as bioinformatics or social networks. The most common learning tasks are node classification, link prediction, community detection, and subgraph similarity analysis.

Classical Machine Learning approaches mainly used *feature engineering* to represent graph information in a representation space. Since the advent of *Representation Learning* approaches, a large body of work has focused on adapting proven techniques from text or image processing to these structured data. The goal is of course to learn a representation of nodes in an *end-to-end* manner, guided by the task. We will cover three approaches in this lab : the first one that only looks locally at the neighborhood of a node to learn the representation, the second *node2vec* which is inspired by *word2vec* to explore the graph topology more deeply, and finally *Graph Convolutional Network* which iteratively builds a sequence of representations based on the length of the paths considered.

We will use data from MovieLens to illustrate the algorithms : it is a list of movies rated by users. We will model a graph between movies in a very simple way by taking into account user ratings. Much more informative models are possible, you are free to adapt the provided code to make the problem treatment more complex and arrive at a real recommendation system problem (for example by modeling a bipartite graph that involves users and movies).

The code for building the graph is provided in the `utils.py` file : each node of the graph is a movie, and the weight of the edge between two nodes represents the weighted *Pointwise Mutual Information* between the two movies : $pmi(x, y) = \log(\frac{x}{D} \frac{y}{D}) - \log(\frac{xy}{D})$ where

- x is the number of times a user rated movie x with a score above a threshold (here set to 5, maximum rating)
- y is the number of times a user rated movie y with a score above the threshold
- xy is the number of times a user rated both movies x and y with scores above the threshold
- D is the number of ratings above the threshold

To avoid the effects of rarely rated movies, the pmi is multiplied by xy . The edge between x and y will therefore have a weight of $xy \times pmi(x, y)$.

The `networkx` module will be useful in this lab to implement graphs. You will particularly need the following functions :

- `graph.neighbors(idx)` which allows you to get the neighborhood of node `idx`
- `graph.nodes` which allows you to get the list of nodes in a graph

1 Triplet loss and local topology representation

In this section, we are not interested in the graph weights but only in its topology : a movie is considered similar to another if an edge connects them. The objective is to find a representation such that similar movies have very close projections and at the same time are far from dissimilar movies.

To build this representation, we can use for example the *Triplet Loss* cost (used in particular for *siamese networks*). This cost uses a triplet of nodes a, p, n : a the anchor, p a positive node i.e. in the neighborhood of a , and n a negative node, i.e. which is not connected to a . The cost function is written as

$$L(a, p, n) = \max(0, d(f_\theta(a), f_\theta(p)) - d(f_\theta(a), f_\theta(n)) + m)$$

where f_θ is the learned representation, d is a distance (euclidean or cosine for example) and m is a positive scalar representing a margin. It aims to push apart unconnected nodes and bring together connected nodes.

Question 1

Write a `TripletDataset` class that returns a triplet of nodes **anchor**, **positive**, **negative** (where the positive and negative are randomly drawn from the neighborhood and outside the neighborhood of the anchor).

Implement the training loop for the triplet loss and learn a representation of the movies. You can visualize the representation with the *t-sne* algorithm (particularly in `tensorboard` with the `add_embedding` function).

For a few movies, observe which movies have the closest representations to them.

One of the major drawbacks of triplet loss is that it requires the negatives to be "well" drawn : if only distant points are chosen, then the cost no longer evolves. It is possible to bias the selection of negatives to only select "useful" negatives in the neighborhood of the anchor.

2 Node2Vec

The *Node2Vec* approach is inspired by *word2vec* in text and more precisely by the *Skip-Gram* model. Very succinctly, this model aims to predict from a word its context (the words around it that are found in the corpus sentences). The training proceeds as follows : given a sentence $t_1 t_2 t_3 \dots t_L$, a target word embedding function f_{in} and a context word embedding function f_{cont} , and a window w , for a given position i the pairs (t_i, t_{i+j}) are constructed with $-w \leq j \leq w$ (and $j \neq 0$). The objective is to maximize $p(t_{i-w}, \dots, t_{i+w} | t_i) = \prod_{-w \leq j \leq w, j \neq 0} p(t_{i+j} | t_i)$, modeled by

$$p(t_{i+j} | t_i) = \frac{e^{f_{in}(t_i) \cdot f_{cont}(t_{i+j})}}{\sum_t e^{f_{in}(t_i) \cdot f_{cont}(t)}}$$

Since optimizing this cost function is impossible due to the softmax over the entire vocabulary, *negative sampling* is used to approximate the cost. In this approximation, the cost function is

$$\sum_{-w \leq j \leq w, j \neq 0} \left[\log(\sigma(f_{in}(t_i) \cdot f_{cont}(t_{i+j}))) + \sum_{k=1}^N \log(\sigma(-f_{in}(t_i) \cdot f_{cont}(\tilde{t}_k))) \right]$$

, where $\tilde{t}_k$ are N words randomly drawn from the vocabulary (assumed to be negative).

To be able to use the Skip-gram model on a graph where nodes would represent tokens, it must be represented in the form of a sequence. The idea is to sample paths randomly from all nodes of the graph and have this set of paths constitute the "sentences" of the corpus : the frequency of appearance of a node in the context of another thus informs about the connectivity between these two nodes and the topology is thus partially taken into account.

The uniform random walk in a graph suffers however from certain limitations depending on the graph topology (central nodes too privileged and redundant in particular), which is why the approach favors a biased random walk allowing to alternate between a breadth-first traversal regime where neighboring nodes are traversed and a depth-first traversal regime that allows moving further away from the initial node. A parameter p allows controlling the probability of returning to the previously visited node (a low value will generate a path that only cycles between two nodes while a large value will prevent any backtracking), and a parameter q allows exploring rather neighboring nodes of already visited nodes or conversely moving away from known nodes (for a low parameter value). The code for this algorithm is provided in `utils.py`.

Question 2

Implement the **Node2Vec** algorithm and analyze the learned representations.

3 Graph Neural Networks (GNN)

The objective of GNNs is to learn a representation of nodes iteratively : starting from an initial representation (which can be a one-hot encoding or a vector representation of the node, such as features describing it - which previous approaches did not allow), a first layer allows learning a new representation by aggregating information from adjacent nodes and itself. This representation is in turn transformed by a second layer, then a third and so on. This process allows taking into account information that comes from nodes increasingly far from the initial node : the first layer only integrates information from the local neighborhood, the second however can take into account the neighborhood of the neighbors of the node and so on. It is an approach inspired by convolutional networks, the information aggregation that a layer does for a node can be likened to a filter centered on this node.

Let h_i^{l-1} be the representation of node i at layer $l - 1$, the generic form to obtain the representation of layer $l + 1$ is as follows :

$$h_i^l = g \left(W_l \sum_{j \in V_i} \frac{h_j^{l-1}}{|V_i|} + B_l h_i^{l-1} \right)$$

where V_i is the neighborhood of node i and g is a non-linearity such as a ReLU for example. The operation on neighbors is equivalent to an average-pooling of their representation.

This equation defines the network model. For its training, any type of method and cost function can be used, for example in our case the same random walk as in the previous section.

A slightly more stable version - the Graph Convolutional Network - normalizes the pooling operation based on the neighborhood :

$$h_i^l = g \left(W_l \sum_{j \in V_i \cup \{i\}} \frac{h_j^{l-1}}{\sqrt{|V_i||V_j|}} \right)$$

and has the advantage of being more efficient to compute using the adjacency matrix A and the diagonal degree matrix D such that $D_{ii} = \sum_j A_{ij}$:

$$H^l = g \left(D^{-\frac{1}{2}} (A + I) D^{-\frac{1}{2}} H^{l-1} W_l \right)$$

Question 3

Implement a GNN and compare the obtained representations.

Note on optimization : The GCN has an important particularity : at each forward pass, it calculates the embeddings of **all nodes** via the operation $D^{-\frac{1}{2}} (A + I) D^{-\frac{1}{2}} H$. Classic mini-batch optimization is therefore inefficient because it recalculates the com-

plete embeddings at each batch.

For efficient training with triplet loss, it is recommended to use full-batch optimization :

1. Calculate the embeddings of all nodes only once per epoch
2. Iterate over batches of triplets with simple indexing of pre-calculated embeddings
3. Accumulate gradients over all triplets
4. A single weight update per epoch

This approach reduces the number of forward passes from $N \times 3$ (where N is the number of batches) to a single forward per epoch, while preserving the computation graph for backpropagation.

4 Model Evaluation and Comparison

To compare models, you can use the following pretext tasks :

Link Prediction Evaluate the model’s ability to predict the existence of edges in the graph. To do this, sample positive (connected) and negative (non-connected) pairs of nodes, calculate their cosine similarity in the embedding space, and measure :

- **Accuracy** : Correct classification rate with an optimal threshold
- **AUC (Area Under Curve)** : Ability to separate positive from negative links
- **Score Separation** : Average difference between positive and negative pair scores

Clustering quality Evaluate the structure of embeddings in the latent space by applying a clustering algorithm (K-Means) and measuring :

- **Silhouette Score** : Intra-cluster cohesion and inter-cluster separation (score close to 1 = well-defined clusters)
- **Modularity** : Quality of the graph partition according to the obtained clusters
- Test different numbers of clusters : $k \in \{5, 10, 15, 20\}$

Qualitative analysis For a few reference movies, identify the k closest movies in the embedding space and observe whether the recommendations are consistent with the genres/themes of the movies.